

Cloud & Big Data

CI/CD pour les pipelines data

Denis AKPAGNONITE

ESG/S Master 1 IA/Big Data

Année universitaire 2025-2026

Où en est la plateforme Anfa

Depuis la séance 1, vous avez construit :

- **Infrastructure** (S1-S4) : Docker, Kubernetes, Terraform.
- **Traitement** (S5) : cluster Spark. **Orchestration** (S6) : Airflow.
- **Temps réel** (S7) : Kafka, flotte de bus en streaming.

Un point commun à tout ce qu'on a fait jusqu'ici : on a toujours déployé à la main. `docker compose up`, `kubectl apply`, dépôt d'un fichier DAG dans un dossier `dags/`. Ça a marché parce qu'on était seul, sur notre propre machine, en TP.

Où en est la plateforme Anfa

Aujourd'hui, on change d'échelle : plusieurs personnes, un vrai serveur de production, et la question devient comment livrer un changement **sans tout casser** ?

Objectifs de la séance

À l'issue de cette séance, vous serez capable de :

1. Expliquer ce qu'est le **DataOps** et pourquoi les pipelines data ont besoin des mêmes disciplines que le développement logiciel.
2. Distinguer les différents **types de tests** applicables à un pipeline de données (unitaires, données, contrats).
3. Décrire l'architecture d'un pipeline **CI/CD** : étapes, déclencheurs, environnements d'exécution.
4. Écrire un **workflow GitHub Actions** qui valide, construit et déploie un pipeline data.
5. Expliquer le principe du **GitOps** et son lien avec l'Infrastructure as Code (S4).

Situation-problème

Mardi 9 h. Mawuli, nouvel arrivant dans l'équipe data d'Anfa, corrige un bug dans le DAG `anfa_pipeline_quotidien` (celui de la séance 6). Il teste rapidement sur sa machine, ça a l'air de marcher. Il copie son fichier corrigé directement dans le dossier `dags/` du serveur Airflow de production.

Deux heures plus tard, le pipeline échoue en pleine nuit. Une erreur de syntaxe qu'il n'avait pas vue localement une variable d'environnement présente sur son laptop mais absente en production. **Personne n'a relu son changement.**

Situation-problème

Le calcul des heures de pointe n'a pas eu lieu cette nuit-là, et personne ne l'a su avant le lendemain matin.

Awa, la responsable technique, demande :

C'est exactement le rôle du CI/CD : Intégration Continue (*Continuous Integration*) et Déploiement Continu (*Continuous Deployment*).

« Comment fait-on pour qu'un changement de code soit automatiquement testé avant d'atteindre la production, et que personne ne puisse déployer sans qu'un collègue ait relu ? »

Le déploiement manuel : ce qu'on a fait jusqu'ici

Récapitulons honnêtement notre pratique des 7 séances passées :

Séance	Comment on a "déployé"
S1-S2	<code>docker compose up</code> à la main
S3	<code>kubectl apply -f manifeste.yaml</code> à la main
S4	<code>terraform apply</code> à la main (mais au moins, versionné)
S6	Copie d'un fichier <code>.py</code> dans le dossier <code>dags/</code> d'Airflow
S7	Lancement manuel d'un script Python et d'un <code>spark-submit</code>

Le déploiement manuel : ce qu'on a fait jusqu'ici

Ça a fonctionné parce qu'on était seul, en environnement de test. Dès qu'on est plusieurs, ou que l'environnement cible est un vrai serveur de production, cette pratique devient dangereuse :

- Pas de **relecture** avant que le changement parte en production.
- Pas de **tests automatiques** pour attraper les erreurs avant qu'elles ne cassent quelque chose.
- Pas de **trace** de qui a déployé quoi, quand, pourquoi.
- Aucune garantie que ce qui marche **sur ma machine** marche **ailleurs**.

DataOps : DevOps appliqué à la donnée

Le monde du développement logiciel a résolu ce problème il y a longtemps avec le **DevOps** : automatiser les tests et les déploiements pour livrer du code fiable, souvent, sans stress.

Le DataOps applique cette même discipline aux pipelines de données : automatiser les tests et les déploiements des DAGs, des jobs Spark, des configurations Airflow pas seulement du code applicatif classique.

DataOps : DevOps appliqué à la donnée

Ce que le DataOps ajoute par rapport au DevOps classique :

- Les pipelines data manipulent des **données**, pas seulement du code : il faut aussi tester la **qualité** et la **structure** des données, pas seulement la syntaxe du code qui les traite.
- Un bug dans un pipeline data peut **corrompre silencieusement** des résultats pendant des jours avant d'être détecté (contrairement à un bug applicatif qui plante immédiatement).
- Les pipelines sont souvent **planifiés** (comme les DAGs Airflow, S6), pas déclenchés par un utilisateur l'erreur peut donc passer inaperçue longtemps.

Les types de tests pour un pipeline data

Trois familles de tests s'appliquent à un pipeline data, chacune répondant à une question différente.

1. Tests unitaires (*Unit tests*)

« *Ma fonction Python fait-elle ce qu'elle est censée faire ?* »

```
def test_calcul_capacite_totale():  
    bus_actifs = [{"capacite": 40}, {"capacite": 60}]  
    assert calculer_capacite_totale(bus_actifs) == 100
```

Les types de tests pour un pipeline data

2. Tests de données (*Data tests*)

« *Les données produites respectent-elles les règles attendues ?* »

Exemples : aucune valeur nulle dans `bus_id`, `capacite > 0`, pas de doublons sur `trajet_id`.

3. Contrats de données (*Data contracts*)

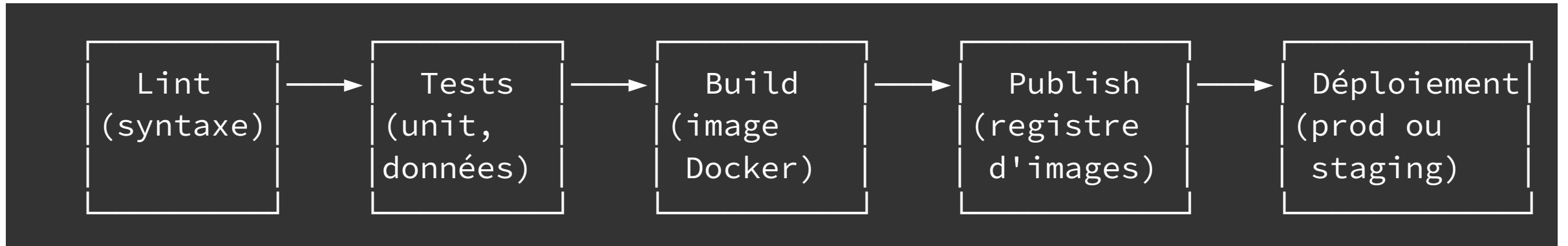
« *Le format de sortie d'une équipe correspond-il à ce que l'équipe suivante attend en entrée ?* »

Les types de tests pour un pipeline data

Exemple : l'équipe Streaming (S7) garantit que chaque message Kafka contient `bus_id`, `ligne_id`, `timestamp` l'équipe qui consomme ce flux peut compter dessus **sans vérifier à chaque fois**.

Anatomie d'un pipeline CI/CD

Un pipeline CI/CD s'organise en **étapes séquentielles**, chacune devant réussir pour passer à la suivante.



Anatomie d'un pipeline CI/CD

- Lint : vérifier le style et la syntaxe du code (ex : `flake8`, `black --check`).
- Tests : exécuter les tests unitaires et de données (ex : `pytest`).
- Build : construire une image Docker si nécessaire (ex : `docker build`).
- Publish : publier l'artefact quelque part (ex : pousser l'image dans un registre)
- Déploiement : livrer le changement en production (ex : copier le DAG validé vers Airflow).

Si une étape échoue, le pipeline s'arrête là. Le code buggé de Mawuli n'aurait jamais atteint la production.

CI vs CD deux disciplines liées mais distinctes

	Intégration Continue (CI)	Déploiement Continu (CD)
Question	Mon changement casse-t-il quelque chose ?	Mon changement peut-il aller en production ?
Déclenchement	À chaque <code>push</code> ou pull request	Après succès de la CI, souvent au merge
Actions	Lint, tests, build	Publication, déploiement effectif

CI vs CD deux disciplines liées mais distinctes

	Intégration Continue (CI)	Déploiement Continu (CD)
Fréquence	Très fréquente (plusieurs fois/jour)	Selon la politique de l'équipe

Nuance importante : *Continuous Deployment* (déploiement automatique, sans intervention humaine) diffère de *Continuous Delivery* (le changement est **prêt** à être déployé, mais un humain valide le déclenchement final).

CI vs CD deux disciplines liées mais distinctes

En pratique, beaucoup d'équipes data choisissent la **Continuous Delivery** pour les pipelines critiques l'automatisation prépare tout, mais un humain clique sur "déployer".

Pour Anfa, on utilisera un modèle de **Continuous Delivery** : le pipeline CI valide tout automatiquement, mais le déploiement en "production" (au sens du TP) se déclenche au merge sur la branche principale **18/41**

GitHub Actions vue d'ensemble

GitHub Actions est la plateforme CI/CD intégrée à GitHub. Elle exécute des **workflows** définis en YAML, stockés directement dans votre dépôt.

Vocabulaire GitHub Actions :

- **Workflow** : un pipeline complet, défini dans un fichier `.github/workflows/*.yaml`.
- **Trigger** (déclencheur) : l'événement qui démarre le workflow (`push`, `pull_request`, planification).
- **Job** : un ensemble d'étapes exécutées sur une même machine virtuelle.

GitHub Actions vue d'ensemble

- **Step** (étape) : une action unique dans un job (exécuter une commande, utiliser une action prête à l'emploi).
- **Runner** : la machine virtuelle qui exécute le job (fournie gratuitement par GitHub pour les dépôts publics).
- **Action** : un bloc réutilisable publié par la communauté (ex : `actions/checkout` pour récupérer le code).

Pourquoi GitHub Actions pour ce cours ? Vos dépôts de rendus sont déjà sur GitHub (portfolio, S1). Aucun outil supplémentaire à installer, aucun compte cloud à créer : les runners sont fournis gratuitement par GitHub sur les dépôts publics.

Tester un DAG Airflow ce qu'on peut réellement vérifier

On ne peut pas lancer Airflow complet dans un test rapide de CI trop lourd, trop lent. Mais on peut tester **beaucoup** sans ça.

1. Le DAG est-il syntaxiquement valide (s'importe sans erreur) ?

```
# tests/test_dag_validate.py
import importlib.util

def test_dag_importable():
    """Le fichier DAG doit pouvoir être importé sans erreur Python."""
    spec = importlib.util.spec_from_file_location(
        "dag_anfa_quotidien", "dags/dag_anfa_quotidien.py"
    )
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)    # échoue si erreur de syntaxe ou d'import
```

Tester un DAG Airflow ce qu'on peut réellement vérifier

2. Les fonctions métier du DAG font-elles ce qu'elles doivent faire ?

```
# tests/test_fonctions_metier.py
from dags.dag_anfa_quotidien import verifier_resultats

def test_verifier_resultats_echoue_si_vide(mock_s3_vide):
    """La vérification doit lever une erreur si aucun fichier n'est trouvé."""
    import pytest
    with pytest.raises(ValueError):
        verifier_resultats()
```

3. Le DAG a-t-il une structure cohérente (pas de cycle, tâches bien nommées) ?

C'est précisément ce que fait le **DAG Integrity Test**, un pattern standard dans l'écosystème Airflow.

Construire et publier une image Docker en CI

Rappel S1-S2 : on construit des images Docker pour livrer nos applications data de façon reproductible. En CI/CD, cette construction devient **automatique**.

Tester un DAG Airflow ce qu'on peut réellement vérifier

```
construire-image:
  runs-on: ubuntu-latest
  needs: valider-dag          # n'exécute ce job QUE si "valider-dag" a réussi

  steps:
    - name: Récupération du code
      uses: actions/checkout@v4

    # Construction de l'image, taguée avec le hash du commit
    - name: Construire l'image Docker
      run: |
        docker build \
          -t anfa-airflow:${{ github.sha }} \
          -f Dockerfile .

    # On pourrait ensuite la publier vers un registre (Docker Hub, GHCR...)
    # Non fait en TP pour rester 100% local, mais voici le principe :
    # - name: Publier l'image
    #   run: docker push anfa-airflow:${{ github.sha }}
```

Tester un DAG Airflow ce qu'on peut réellement vérifier

À comprendre.

- `needs: valider-dag` : crée une **dépendance entre jobs**. Si les tests échouent, l'image n'est même pas construite. On économise du temps et on évite de publier du code cassé.
- `${{ github.sha }}` : variable fournie par GitHub Actions, contenant le hash du commit. Tagger l'image avec ce hash garantit une **traçabilité totale** : on sait exactement quel commit a produit quelle image.

Le déploiement livrer le changement

Une fois les tests et le build validés, il faut **déployer** c'est-à-dire rendre le changement effectif sur l'environnement cible.

Trois approches courantes, de la plus simple à la plus robuste :

Approche	Principe	Limite
Copie de fichiers	Le pipeline copie le DAG validé vers le dossier <code>dags/</code> d'Airflow	Simple, mais peu robuste à grande échelle

Le déploiement livrer le changement

Approche	Principe	Limite
Synchronisation Git (<i>git-sync</i>)	Un processus surveille en continu une branche Git et synchronise automatiquement	Standard dans les déploiements Airflow en production
Redéploiement d'image	Toute la stack (avec le nouveau DAG inclus) est reconstruite et redéployée	Le plus robuste, aligné avec les principes de l'IaC (S4)

Le déploiement livrer le changement

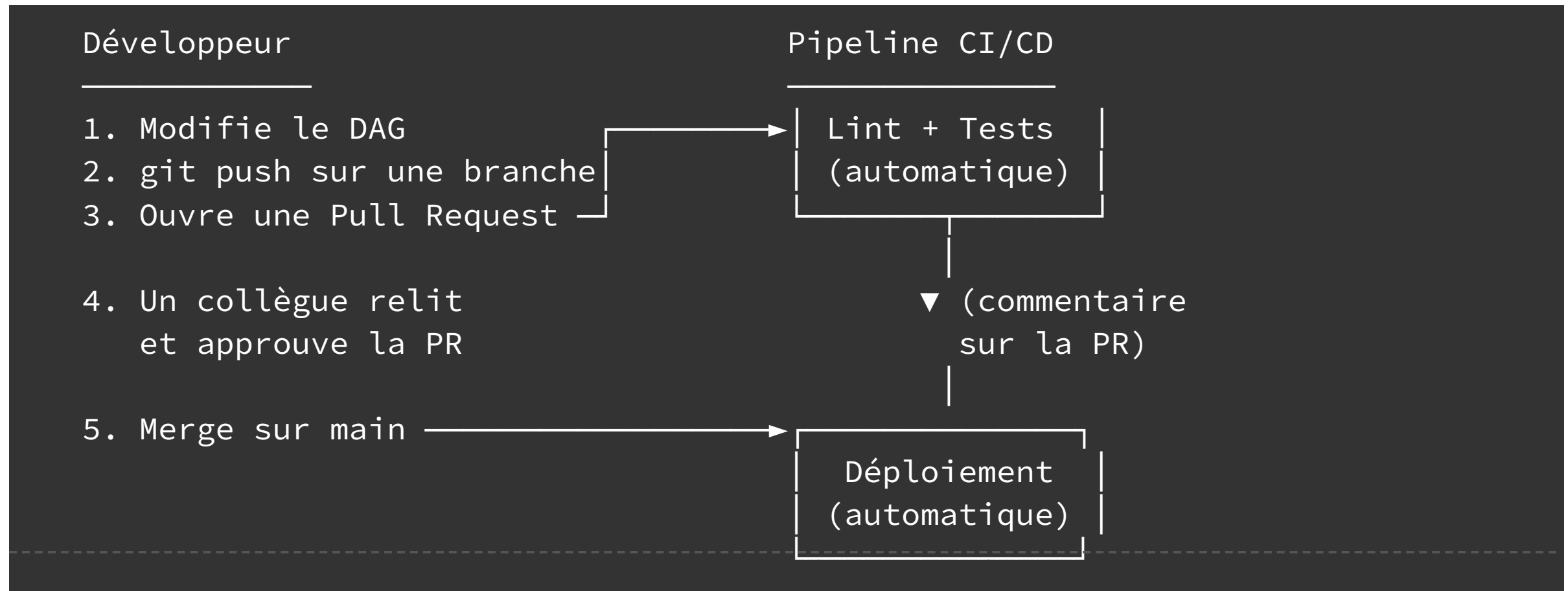
Pour ce TP, on utilise l'approche la plus simple (copie de fichiers) pour rester pédagogique et rapide mais retenez que les deux autres existent et sont la norme en production réelle.

```
deployer:
  runs-on: ubuntu-latest
  needs: construire-image
  if: github.ref == 'refs/heads/main'    # seulement si on est sur main (pas sur une PR)

  steps:
    - name: "Déployer" le DAG (simulation locale pour le TP)
      run: cp dags/*.py chemin/vers/airflow/dags/
```

GitOps le pont avec l'Infrastructure as Code

Rappel S4 : nous avons évoqué le **workflow GitOps** en préfigurant cette séance. Le voici en détail.



Aucun changement n'est appliqué manuellement tout passe par un commit, une revue, un merge.

GitOps le pont avec l'Infrastructure as Code

Ce que ça change pour Mawuli : son code buggé aurait été **bloqué à l'étape des tests**, visible dans la Pull Request, **avant** qu'un collègue n'accepte de merger. Aucune chance qu'il atteigne la production sans validation.

Les avantages du GitOps, concrètement

- **Aucune action manuelle en production** : personne ne se connecte en SSH pour modifier un fichier à la main.
- **Traçabilité complète** : chaque changement est un commit, avec un auteur, une date, un message.
- **Réversibilité** : `git revert` permet de revenir instantanément à un état antérieur qui fonctionnait.

Les avantages du GitOps, concrètement

- **Revue systématique** : une Pull Request impose qu'un collègue regarde le changement avant qu'il ne parte en production.
- **Cohérence** : ce qui est déployé correspond **exactement** à ce qui est dans Git pas de dérive entre "ce qu'on croit avoir déployé" et "ce qui tourne réellement".

Ce principe traverse tout ce que vous avez appris : le state Terraform (S4) qui doit rester synchronisé avec le code, le DAG Airflow versionné (S6), et maintenant le pipeline CI/CD qui automatise le passage de l'un à l'autre. **Le CI/CD est le moteur qui fait tourner le GitOps.**

Équivalences cloud CI/CD partout

Outil open source	AWS	GCP	Azure
GitHub Actions (notre TP)	CodePipeline + CodeBuild	Cloud Build	Azure Pipelines
Registre d'images	ECR	Artifact Registry	Azure Container Registry

Équivalences cloud CI/CD partout

Outil open source	AWS	GCP	Azure
Runners auto-hébergés	EC2 / self-hosted runners	Compute Engine	Azure VMs

Autres plateformes CI/CD populaires (hors cloud spécifique) :
GitLab CI, Jenkins (historique, très répandu en entreprise), CircleCI.

Ce que vous apprenez avec GitHub Actions est directement transposable. Le vocabulaire change (job/stage, runner/agent), mais la logique déclencheur, étapes séquentielles, dépendances entre jobs est universelle.

Points clés à retenir

1. **Le déploiement manuel** (ce qu'on a fait de S1 à S7) devient dangereux dès qu'on est plusieurs ou qu'on vise un vrai environnement de production.
2. Le **DataOps** applique au pipeline data la même discipline que le DevOps applique au code : automatiser tests et déploiements.
3. Trois familles de tests pour un pipeline data : **tests unitaires** (le code), **tests de données** (la qualité), **contrats de données** (l'interface entre équipes).
4. Un pipeline CI/CD s'organise en étapes séquentielles : **lint** → **tests** → **build** → **publish** → **déploiement**, chacune bloquant la suivante en cas d'échec.

Points clés à retenir

5. **CI** (intégration continue) valide qu'un changement ne casse rien ; **CD** (déploiement continu) le rend disponible en production, automatiquement ou après validation humaine.
6. **GitHub Actions** exécute des **workflows** YAML déclenchés par des événements Git (`push`, `pull_request`).
7. Le **GitOps** fait de Git la source unique de vérité : aucun changement manuel, tout passe par une Pull Request revue et un merge.

Vocabulaire de la séance 8

Terme français	Terme anglais	Définition
Intégration continue	<i>Continuous Integration (CI)</i>	Pratique consistant à tester automatiquement chaque changement de code.
Déploiement continu	<i>Continuous Deployment/Delivery (CD)</i>	Pratique consistant à livrer automatiquement (ou semi-automatiquement) un changement validé.

Vocabulaire de la séance 8

Terme français	Terme anglais	Définition
Flux de travail	<i>Workflow</i>	Pipeline CI/CD défini en YAML dans GitHub Actions.
Déclencheur	<i>Trigger</i>	Événement qui démarre un workflow (push, pull request, planification).
Exécuteur	<i>Runner</i>	Machine virtuelle qui exécute les étapes d'un job.

Vocabulaire de la séance 8

Terme français	Terme anglais	Définition
Contrat de données	<i>Data contract</i>	Accord formel sur le format des données échangées entre deux équipes ou systèmes.
Opérations sur la donnée	<i>DataOps</i>	Application des principes DevOps aux pipelines de données.
Git comme source de vérité	<i>GitOps</i>	Pratique où tout changement d'infrastructure ou de pipeline passe exclusivement par Git.

Place au TP

Écrire et déclencher un pipeline CI/CD qui valide et déploie un DAG Anfa